

Leva.Framework.Identity.Memory

`Leva.Framework.Identity.Memory` is the in-memory provider implementation of `Leva.Framework.Identity`. It stores principals and auth sessions in process and authenticates principal names for tests, demos, samples, and early applications.

Memory identity exists for tests, demos, samples, and short-lived local application scenarios where durable identity storage is not required. It depends on `Leva.Framework.Identity` and `Leva.Framework.Core`; it does not depend on Engine, Storage, ASP.NET Core, databases, password hashing packages, or external identity providers.

```
Leva.Framework.Identity.Memory
    -> Leva.Framework.Core
    -> Leva.Framework.Identity
```

Project overview

The memory provider is intentionally simple. `MemoryPrincipalStore` keeps principals in memory. `MemoryAuthSessionStore` keeps auth sessions in memory. `MemoryAuthenticationPolicy` authenticates a submitted name by resolving it through `IPrincipalStore` using the `memory.principal` method. It does not store passwords, secrets, reset tokens, email confirmation state, lockout state, or account lifecycle data.

Authorization is not memory-specific. `MemoryPrincipalServices` uses the shared `BuiltInAuthorizationPolicy` from `Leva.Framework.Identity` for signed-in, role, permission, and claim checks. `MemoryAuthSessionSource` exposes one configured current auth session for tests, demos, and simple hosts.

`MemoryPrincipalServices` is a small composition helper. It creates the memory stores, authentication policy, shared authorization policy, auth session service, authentication service, authorization service, auth session source, and principal access object while still leaving each individual object available for direct testing or manual wiring.

Files and classes

Provider composition

`MemoryPrincipalServices` - Groups the memory stores, policies, services, auth session source, and principal access object for quick setup.

Principal storage

`MemoryPrincipalStore` - Stores principals in memory and resolves them by id, display name, email, or configured names.

Auth session implementation

`MemoryAuthSessionStore` - Stores auth sessions in memory and implements `IAuthSessionStore`. `MemoryAuthSessionSource` - Resolves the current auth session from a configured in-memory session id.

Authentication implementation

`MemoryAuthenticationMethods` - Defines the standard `memory.principal` authentication method. `MemoryAuthenticationPolicy` - Authenticates submitted names by resolving them through `IPrincipalStore`.